

Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «УрФУ имени первого Президента России Б.Н. Ельцина»
Институт радиоэлектроники и информационных технологий – РТФ

«Система ввода/вывода в Java. Работа с файлами»

Лабораторное задание №7

Группа: РИМ-150950

Студент: Владимиров Н.А.

Преподаватель: Архипов Н. А.

Екатеринбург

2026

1. Цель работы

Получить навыки работы с каталогами и файлами операционной системы, а также с классами ввода/вывода Java. Научиться выполнять ввод/вывод данных файла через байтовые и символьные потоки, использовать буферизацию, сериализацию объектов и класс `PrintWriter`.

2. Описание задачи

В разделе 1 методички приведены примеры:

1. работа с классом `File`;
2. байтовый ввод/вывод (`FileInputStream`, `FileOutputStream`);
3. символьные потоки (`FileReader`, `FileWriter`);
4. буферизация (`BufferedReader`, `BufferedWriter`);
5. преобразование байтовых потоков в символьные (`InputStreamReader`, `OutputStreamWriter`);
6. запись в файл с помощью `PrintWriter`;
7. сериализация объекта `Person`;
8. подсчёт строк в файле и копирование файла.

В разделе 2 необходимо выполнить задания для самостоятельной работы:

1. воспроизвести все примеры из раздела 1;
2. доработать примеры для ввода данных с консоли;
3. подсчитать количество строк в текстовом файле;
4. скопировать содержимое одного файла в другой;

5. вывести размер файла в байтах;
6. найти строки, содержащие заданное слово;
7. записать текст в файл и вывести количество записанных символов;
8. сериализовать и десериализовать объект класса Student;
9. решить две задачи с сайта timus.ru

3. Ход выполнения

Создан Java-проект lab_7. Примеры из раздела 1 размещены в пакете examples_lr7 (классы Example1–Example9). Задания 1–8 реализованы в пакетах lr7.task1–lr7.task8. Вспомогательный класс FileDemoConfig задаёт каталог data для демонстрационных файлов. Запуск примеров и заданий выполняется через класс Main.

Репозиторий с исходным кодом: https://github.com/n0vision/Java_Labs_2026

3.1. Примеры из раздела 1

Пример 1 (класс examples_lr7.Example1):

```
package examples_lr7;

import lr7.FileDemoConfig;

import java.io.File;
import java.io.IOException;

public class Example1 {

    public static void run() throws IOException {
        File folder = FileDemoConfig.resolve("example1",
"example_folder").toFile();
        File file = new File(folder, "example_file.txt");

        System.out.println("Создание папки: " + folder.getPath());
        if (!folder.exists()) {
            if (folder.mkdirs()) {
                System.out.println("Папка создана.");
            } else {
                System.out.println("Не удалось создать папку.");
            }
        }
    }
}
```

```

    } else {
        System.out.println("Папка уже существует.");
    }

    System.out.println("Создание файла: " + file.getPath());
    if (!file.exists()) {
        if (file.createNewFile()) {
            System.out.println("Файл создан.");
        } else {
            System.out.println("Не удалось создать файл.");
        }
    } else {
        System.out.println("Файл уже существует.");
    }

    System.out.println("Имя файла: " + file.getName());
    System.out.println("Абсолютный путь: " + file.getAbsolutePath());
    System.out.println("Размер файла: " + file.length() + " байт");
    System.out.println("Это файл: " + file.isFile());
    System.out.println("Это каталог: " + file.isDirectory());

    System.out.println("Удаление файла...");
    if (file.exists() && file.delete()) {
        System.out.println("Файл удалён.");
    }

    System.out.println("Удаление папки...");
    if (folder.exists() && folder.delete()) {
        System.out.println("Папка удалена.");
    }
}

public static void main(String[] args) throws IOException {
    System.out.println("Пример 1.1. Работа с классом File");
    run();
}
}

```

Пример 2 (класс examples_lr7.Example2):

```

package examples_lr7;

import lr7.FileDemoConfig;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.nio.file.Files;

public class Example2 {

    public static void run(String data) throws IOException {
        var file = FileDemoConfig.resolve("example2", "example_file.txt");
        Files.createDirectories(file.getParent());

        System.out.println("Запись данных в файл: \"" + data + "\"");
        try (FileOutputStream output = new FileOutputStream(file.toFile())) {
            output.write(data.getBytes());
            System.out.println("Данные записаны.");
        }
    }
}

```

```

    }

    System.out.println("Чтение данных из файла:");
    try (FileInputStream input = new FileInputStream(file.toFile())) {
        int symbol;
        StringBuilder builder = new StringBuilder();
        while ((symbol = input.read()) != -1) {
            builder.append((char) symbol);
        }
        System.out.println("Прочитано: " + builder);
    }

    System.out.println("Удаление файла...");
    if (Files.deleteIfExists(file)) {
        System.out.println("Файл удалён.");
    }
}

public static void main(String[] args) throws IOException {
    System.out.println("Пример 1.2. Байтовый ввод и вывод (FileInputStream /
FileOutputStream)");
    run("data");
}
}

```

Пример 3 (класс examples_lr7.Example3):

```

package examples_lr7;

import lr7.FileDemoConfig;

import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.nio.file.Files;

public class Example3 {

    public static void run(String data) throws IOException {
        var file = FileDemoConfig.resolve("example3", "example_file.txt");
        Files.createDirectories(file.getParent());

        System.out.println("Запись данных в файл: \"" + data + "\"");
        try (FileWriter writer = new FileWriter(file.toFile())) {
            writer.write(data);
            System.out.println("Данные записаны.");
        }

        System.out.println("Чтение данных из файла:");
        try (FileReader reader = new FileReader(file.toFile())) {
            int symbol;
            StringBuilder builder = new StringBuilder();
            while ((symbol = reader.read()) != -1) {
                builder.append((char) symbol);
            }
            System.out.println("Прочитано: " + builder);
        }

        System.out.println("Удаление файла...");
    }
}

```

```

        if (Files.deleteIfExists(file)) {
            System.out.println("Файл удалён.");
        }
    }

    public static void main(String[] args) throws IOException {
        System.out.println("Пример 1.3. Символьные потоки (FileReader /
FileWriter)");
        run("data");
    }
}

```

Пример 4 (класс examples_lr7.Example4):

```

package examples_lr7;

import lr7.FileDemoConfig;

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.nio.file.Files;

public class Example4 {

    public static void run(String data) throws IOException {
        var file = FileDemoConfig.resolve("example4", "example_file.txt");
        Files.createDirectories(file.getParent());

        System.out.println("Запись данных в файл: \"\" + data + "\"");
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(file.toFile())) {
            writer.write(data);
            System.out.println("Данные записаны.");
        }

        System.out.println("Чтение данных из файла:");
        try (BufferedReader reader = new BufferedReader(new
FileReader(file.toFile())) {
            String line;
            StringBuilder builder = new StringBuilder();
            while ((line = reader.readLine()) != null) {
                builder.append(line);
            }
            System.out.println("Прочитано: " + builder);
        }

        System.out.println("Удаление файла...");
        if (Files.deleteIfExists(file)) {
            System.out.println("Файл удалён.");
        }
    }

    public static void main(String[] args) throws IOException {
        System.out.println("Пример 1.4. Буферизация (BufferedReader /
BufferedWriter)");
        run("data");
    }
}

```

```
    }  
}
```

Пример 5 (класс examples_lr7.Example5):

```
package examples_lr7;  
  
import lr7.FileDemoConfig;  
  
import java.io.BufferedReader;  
import java.io.BufferedWriter;  
import java.io.FileInputStream;  
import java.io.FileOutputStream;  
import java.io.IOException;  
import java.io.InputStreamReader;  
import java.io.OutputStreamWriter;  
import java.nio.charset.StandardCharsets;  
import java.nio.file.Files;  
  
public class Example5 {  
  
    public static void run() throws IOException {  
        var inputFile = FileDemoConfig.resolve("example5", "input.txt");  
        var outputFile = FileDemoConfig.resolve("example5", "output.txt");  
        Files.createDirectories(inputFile.getParent());  
  
        String sourceText = "первая строка\нвторая строка\нтретья строка";  
        Files.writeString(inputFile, sourceText, StandardCharsets.UTF_8);  
        System.out.println("Исходный файл input.txt создан.");  
  
        System.out.println("Чтение из input.txt и преобразование в верхний  
регистр...");  
        try (BufferedReader reader = new BufferedReader(  
            new InputStreamReader(new FileInputStream(inputFile.toFile()),  
StandardCharsets.UTF_8));  
            BufferedWriter writer = new BufferedWriter(  
                new OutputStreamWriter(new  
FileOutputStream(outputFile.toFile()), StandardCharsets.UTF_8))) {  
  
            String line;  
            while ((line = reader.readLine()) != null) {  
                writer.write(line.toUpperCase());  
                writer.newLine();  
            }  
        }  
  
        System.out.println("Результат записан в output.txt:");  
        System.out.println(Files.readString(outputFile,  
StandardCharsets.UTF_8));  
  
        Files.deleteIfExists(inputFile);  
        Files.deleteIfExists(outputFile);  
        System.out.println("Временные файлы удалены.");  
    }  
  
    public static void main(String[] args) throws IOException {  
        System.out.println("Пример 1.5. Преобразование байтовых потоков в  
символьные");  
        run();  
    }  
}
```

```
    }  
}
```

Пример 6 (класс examples_lr7.Example6):

```
package examples_lr7;  
  
import lr7.FileDemoConfig;  
  
import java.io.IOException;  
import java.io.PrintWriter;  
import java.nio.charset.StandardCharsets;  
import java.nio.file.Files;  
  
public class Example6 {  
  
    public static void run() throws IOException {  
        var file = FileDemoConfig.resolve("example6", "print_writer.txt");  
        Files.createDirectories(file.getParent());  
  
        System.out.println("Запись в файл с помощью PrintWriter...");  
        try (PrintWriter writer = new PrintWriter(file.toFile(),  
StandardCharsets.UTF_8)) {  
            writer.print("Строка без перевода. ");  
            writer.println("Строка с переводом.");  
            writer.printf("Число: %d, дробь: %.2f\n", 42, 3.1415);  
        }  
  
        System.out.println("Содержимое файла:");  
        System.out.println(Files.readString(file, StandardCharsets.UTF_8));  
  
        System.out.println("Вывод на консоль через PrintWriter(System.out):");  
        try (PrintWriter consoleWriter = new PrintWriter(System.out, true,  
StandardCharsets.UTF_8)) {  
            consoleWriter.println("Сообщение через PrintWriter");  
        }  
  
        Files.deleteIfExists(file);  
        System.out.println("Файл удалён.");  
    }  
  
    public static void main(String[] args) throws IOException {  
        System.out.println("Пример 1.6. Класс PrintWriter");  
        run();  
    }  
}
```

Пример 7 (класс examples_lr7.Example7):

```
package examples_lr7;  
  
import lr7.FileDemoConfig;  
  
import java.io.FileInputStream;  
import java.io.FileOutputStream;  
import java.io.IOException;  
import java.io.ObjectInputStream;  
import java.io.ObjectOutputStream;
```

```

import java.nio.file.Files;

public class Example7 {

    public static void run() throws IOException, ClassNotFoundException {
        var file = FileDemoConfig.resolve("example7", "person.ser");
        Files.createDirectories(file.getParent());

        Person person = new Person("Иван", 25);
        System.out.println("Исходный объект: " + person);

        System.out.println("Сериализация объекта в файл...");
        try (ObjectOutputStream output = new ObjectOutputStream(new
FileOutputStream(file.toFile())) {
            output.writeObject(person);
        }

        System.out.println("Десериализация объекта из файла...");
        Person restored;
        try (ObjectInputStream input = new ObjectInputStream(new
FileInputStream(file.toFile())) {
            restored = (Person) input.readObject();
        }

        System.out.println("Восстановленный объект: " + restored);

        Files.deleteIfExists(file);
        System.out.println("Файл сериализации удалён.");
    }

    public static void main(String[] args) throws IOException,
ClassNotFoundException {
        System.out.println("Пример 1.7. Сериализация объекта Person");
        run();
    }
}

```

Пример 8 (класс examples_lr7.Example8):

```

package examples_lr7;

import lr7.FileDemoConfig;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;

public class Example8 {

    public static int countLines(String filePath) throws IOException {
        int lines = 0;
        try (BufferedReader reader = new BufferedReader(new
FileReader(filePath)) {
            while (reader.readLine() != null) {
                lines++;
            }
        }
    }
}

```

```

        return lines;
    }

    public static void run() throws IOException {
        var file = FileDemoConfig.resolve("example8", "lines.txt");
        Files.createDirectories(file.getParent());
        Files.writeString(file, "строка 1\nстрока 2\nстрока 3\n",
StandardCharsets.UTF_8);

        int lineCount = countLines(file.toString());
        System.out.println("Файл: " + file);
        System.out.println("Количество строк: " + lineCount);

        Files.deleteIfExists(file);
    }

    public static void main(String[] args) throws IOException {
        System.out.println("Задача 3. Подсчёт строк в текстовом файле");
        run();
    }
}

```

Пример 9 (класс examples_lr7.Example9):

```

package examples_lr7;

import lr7.FileDemoConfig;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.StandardCopyOption;

public class Example9 {

    public static void copyFile(String sourcePath, String targetPath) throws
IOException {
        Files.copy(
            FileDemoConfig.getDataDir().resolve(sourcePath),
            FileDemoConfig.getDataDir().resolve(targetPath),
            StandardCopyOption.REPLACE_EXISTING
        );
    }

    public static void run() throws IOException {
        var source = FileDemoConfig.resolve("example9", "source.txt");
        var target = FileDemoConfig.resolve("example9", "target.txt");
        Files.createDirectories(source.getParent());
        Files.writeString(source, "Текст для копирования.\nВторая строка.",
StandardCharsets.UTF_8);

        System.out.println("Исходный файл: " + source);
        copyFile("example9/source.txt", "example9/target.txt");
        System.out.println("Файл скопирован в: " + target);
        System.out.println("Содержимое копии:");
        System.out.println(Files.readString(target, StandardCharsets.UTF_8));

        Files.deleteIfExists(source);
        Files.deleteIfExists(target);
    }
}

```

```

    }

    public static void main(String[] args) throws IOException {
        System.out.println("Задача 4. Копирование текстового файла");
        run();
    }
}

```

3.2. Класс Person (сериализация)

```

package examples_lr7;

import java.io.Serial;
import java.io.Serializable;

public class Person implements Serializable {

    @Serial
    private static final long serialVersionUID = 1L;

    private final String name;
    private final int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    @Override
    public String toString() {
        return "Person{name='" + name + "', age=" + age + "}";
    }
}

```

3.3. Задания для самостоятельной работы

Задание 1 (класс lr7.task1.Task1):

```

package lr7.task1;

import examples_lr7.Example1;
import examples_lr7.Example2;
import examples_lr7.Example3;
import examples_lr7.Example4;
import examples_lr7.Example5;
import examples_lr7.Example6;
import examples_lr7.Example7;

public class Task1 {

```

```

    public static void run() throws Exception {
        Example1.run();
        System.out.println();

        Example2.run("data");
        System.out.println();

        Example3.run("data");
        System.out.println();

        Example4.run("data");
        System.out.println();

        Example5.run();
        System.out.println();

        Example6.run();
        System.out.println();

        Example7.run();
    }

    public static void main(String[] args) throws Exception {
        System.out.println("Задание 1. Воспроизведение всех примеров из раздела
1");
        run();
    }
}

```

Задание 2 (класс lr7.task2.Task2):

```

package lr7.task2;

import lr7.FileDemoConfig;

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.util.Scanner;

public class Task2 {

    private static String readConsoleData(Scanner scanner) {
        if (FileDemoConfig.isDemoMode()) {
            String demoData = "demo data from console";
            System.out.println("Введите данные для записи в файл (демо): " +
demoData);
            return demoData;
        }
    }
}

```

```

        System.out.print("Введите данные для записи в файл: ");
        return scanner.nextLine();
    }

    public static void runByteStreams(Scanner scanner) throws Exception {
        String data = readConsoleData(scanner);
        var file = FileDemoConfig.resolve("task2", "byte_file.txt");
        Files.createDirectories(file.getParent());

        try (FileOutputStream output = new FileOutputStream(file.toFile())) {
            output.write(data.getBytes(StandardCharsets.UTF_8));
        }

        try (FileInputStream input = new FileInputStream(file.toFile())) {
            System.out.println("Прочитано из файла: " + new
String(input.readAllBytes(), StandardCharsets.UTF_8));
        }

        Files.deleteIfExists(file);
    }

    public static void runCharStreams(Scanner scanner) throws Exception {
        String data = readConsoleData(scanner);
        var file = FileDemoConfig.resolve("task2", "char_file.txt");
        Files.createDirectories(file.getParent());

        try (FileWriter writer = new FileWriter(file.toFile())) {
            writer.write(data);
        }

        try (FileReader reader = new FileReader(file.toFile())) {
            StringBuilder builder = new StringBuilder();
            int symbol;
            while ((symbol = reader.read()) != -1) {
                builder.append((char) symbol);
            }
            System.out.println("Прочитано из файла: " + builder);
        }

        Files.deleteIfExists(file);
    }

    public static void runBufferedStreams(Scanner scanner) throws Exception {
        String data = readConsoleData(scanner);
        var file = FileDemoConfig.resolve("task2", "buffered_file.txt");
        Files.createDirectories(file.getParent());

        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(file.toFile())))) {
            writer.write(data);
        }

        try (BufferedReader reader = new BufferedReader(new
FileReader(file.toFile())))) {
            System.out.println("Прочитано из файла: " + reader.readLine());
        }

        Files.deleteIfExists(file);
    }
}

```

```

    public static void runStreamAdapter(Scanner scanner) throws Exception {
        String data = readConsoleData(scanner);
        var inputFile = FileDemoConfig.resolve("task2", "adapter_input.txt");
        var outputFile = FileDemoConfig.resolve("task2", "adapter_output.txt");
        Files.createDirectories(inputFile.getParent());
        Files.writeString(inputFile, data, StandardCharsets.UTF_8);

        try (BufferedReader reader = new BufferedReader(
            new InputStreamReader(new FileInputStream(inputFile.toFile()),
            StandardCharsets.UTF_8));
            BufferedWriter writer = new BufferedWriter(
            new OutputStreamWriter(new
            FileOutputStream(outputFile.toFile()), StandardCharsets.UTF_8))) {

            String line;
            while ((line = reader.readLine()) != null) {
                writer.write(line.toUpperCase());
                writer.newLine();
            }

            System.out.println("Преобразованный текст: " +
            Files.readString(outputFile, StandardCharsets.UTF_8));

            Files.deleteIfExists(inputFile);
            Files.deleteIfExists(outputFile);
        }

        public static void runPrintWriter(Scanner scanner) throws Exception {
            String data = readConsoleData(scanner);
            var file = FileDemoConfig.resolve("task2", "print_file.txt");
            Files.createDirectories(file.getParent());

            try (PrintWriter writer = new PrintWriter(file.toFile(),
            StandardCharsets.UTF_8)) {
                writer.println(data);
            }

            System.out.println("Записано в файл: " + Files.readString(file,
            StandardCharsets.UTF_8));
            Files.deleteIfExists(file);
        }

        public static void run() throws Exception {
            try (Scanner scanner = new Scanner(System.in, StandardCharsets.UTF_8)) {
                System.out.println("--- Байтовые потоки ---");
                runByteStreams(scanner);
                System.out.println();

                System.out.println("--- Символьные потоки ---");
                runCharStreams(scanner);
                System.out.println();

                System.out.println("--- Буферизованные потоки ---");
                runBufferedStreams(scanner);
                System.out.println();

                System.out.println("--- Адаптеры потоков ---");
            }
        }
    }

```

```

        runStreamAdapter(scanner);
        System.out.println();

        System.out.println("--- PrintWriter ---");
        runPrintWriter(scanner);
    }
}

public static void main(String[] args) throws Exception {
    System.out.println("Задание 2. Ввод данных с консоли для записи в
файл");
    run();
}
}

```

Задание 3 (класс lr7.task3.Task3):

```

package lr7.task3;

import examples_lr7.Example8;
import lr7.FileDemoConfig;

import java.nio.charset.StandardCharsets;
import java.nio.file.Files;

public class Task3 {

    public static void run() throws Exception {
        var file = FileDemoConfig.resolve("task3", "sample.txt");
        Files.createDirectories(file.getParent());
        Files.writeString(file, "первая\nвторая\nтретья\n",
StandardCharsets.UTF_8);

        int lines = Example8.countLines(file.toString());
        System.out.println("Файл: " + file.getFileName());
        System.out.println("Количество строк: " + lines);

        Files.deleteIfExists(file);
    }

    public static void main(String[] args) throws Exception {
        System.out.println("Задание 3. Подсчёт строк в текстовом файле");
        run();
    }
}

```

Задание 4 (класс lr7.task4.Task4):

```

package lr7.task4;

import examples_lr7.Example9;
import lr7.FileDemoConfig;

import java.nio.charset.StandardCharsets;
import java.nio.file.Files;

public class Task4 {

```

```

    public static void run() throws Exception {
        var source = FileDemoConfig.resolve("task4", "source.txt");
        var target = FileDemoConfig.resolve("task4", "copy.txt");
        Files.createDirectories(source.getParent());
        Files.writeString(source, "Содержимое исходного файла.\nЕщё одна строка.", StandardCharsets.UTF_8);

        Example9.copyFile("task4/source.txt", "task4/copy.txt");
        System.out.println("Файл скопирован.");
        System.out.println(Files.readString(target, StandardCharsets.UTF_8));

        Files.deleteIfExists(source);
        Files.deleteIfExists(target);
    }

    public static void main(String[] args) throws Exception {
        System.out.println("Задание 4. Копирование текстового файла");
        run();
    }
}

```

Задание 5 (класс lr7.task5.Task5):

```

package lr7.task5;

import lr7.FileDemoConfig;

import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.util.Scanner;

public class Task5 {

    public static void run() throws Exception {
        var file = FileDemoConfig.resolve("task5", "size_demo.txt");
        Files.createDirectories(file.getParent());
        Files.writeString(file, "Пример файла для определения размера.", StandardCharsets.UTF_8);

        String fileName;
        try (Scanner scanner = new Scanner(System.in, StandardCharsets.UTF_8)) {
            if (FileDemoConfig.isDemoMode()) {
                fileName = "task5/size_demo.txt";
                System.out.println("Введите имя файла (демо): " + fileName);
            } else {
                System.out.print("Введите имя файла (относительно папки data): ");
                fileName = scanner.nextLine().trim();
            }
        }

        var target = FileDemoConfig.getDataDir().resolve(fileName);
        if (!Files.exists(target)) {
            System.out.println("Файл не найден: " + target);
            return;
        }

        long size = Files.size(target);
        System.out.println("Размер файла \"" + target.getFileName() + "\": " +

```

```

size + " байт");

    Files.deleteIfExists(file);
}

public static void main(String[] args) throws Exception {
    System.out.println("Задание 5. Размер файла в байтах");
    run();
}
}

```

Задание 6 (класс lr7.task6.Task6):

```

package lr7.task6;

import lr7.FileDemoConfig;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Task6 {

    public static List<String> findLinesWithWord(String filePath, String word)
throws IOException {
        List<String> result = new ArrayList<>();
        try (BufferedReader reader = new BufferedReader(new
FileReader(filePath))) {
            String line;
            while ((line = reader.readLine()) != null) {
                if (line.contains(word)) {
                    result.add(line);
                }
            }
        }
        return result;
    }

    public static void run() throws Exception {
        var file = FileDemoConfig.resolve("task6", "search.txt");
        Files.createDirectories(file.getParent());
        Files.writeString(file,
            "Java — язык программирования.\nРабота с файлами в Java.\nПотоки
ввода-вывода.\n",
            StandardCharsets.UTF_8);

        String fileName;
        String word;
        try (Scanner scanner = new Scanner(System.in, StandardCharsets.UTF_8)) {
            if (FileDemoConfig.isDemoMode()) {
                fileName = "task6/search.txt";
                word = "Java";
                System.out.println("Введите имя файла (демо): " + fileName);
                System.out.println("Введите слово для поиска (демо): " + word);
            }
        }
    }
}

```

```

        } else {
            System.out.print("Введите имя файла (относительно папки data):");
        }

        fileName = scanner.nextLine().trim();
        System.out.print("Введите слово для поиска: ");
        word = scanner.nextLine().trim();
    }
}

var target = FileDemoConfig.getDataDir().resolve(fileName);
List<String> lines = findLinesWithWord(target.toString(), word);

System.out.println("Строки, содержащие \"" + word + "\"");
if (lines.isEmpty()) {
    System.out.println("нет совпадений");
} else {
    lines.forEach(System.out::println);
}

Files.deleteIfExists(file);
}

public static void main(String[] args) throws Exception {
    System.out.println("Задание 6. Поиск строк, содержащих заданное слово");
    run();
}
}

```

Задание 7 (класс lr7.task7.Task7):

```

package lr7.task7;

import lr7.FileDemoConfig;

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.util.Scanner;

public class Task7 {

    public static void run() throws Exception {
        String fileName;
        String text;
        try (Scanner scanner = new Scanner(System.in, StandardCharsets.UTF_8)) {
            if (FileDemoConfig.isDemoMode()) {
                fileName = "task7/output.txt";
                text = "Текст для записи в файл.";
                System.out.println("Введите имя файла (демо): " + fileName);
                System.out.println("Введите текст (демо): " + text);
            } else {
                System.out.print("Введите имя файла (относительно папки data):");
            }

            fileName = scanner.nextLine().trim();
            System.out.print("Введите текст для записи: ");
            text = scanner.nextLine();
        }
    }
}

```

```

        var file = FileDemoConfig.getDataDir().resolve(fileName);
        Files.createDirectories(file.getParent());

        try (BufferedWriter writer = new BufferedWriter(new
        FileWriter(fileToFile(), StandardCharsets.UTF_8))) {
            writer.write(text);
        }

        int charCount = text.length();
        System.out.println("Записано символов: " + charCount);
        System.out.println("Содержимое файла: " + Files.readString(file,
        StandardCharsets.UTF_8));

        Files.deleteIfExists(file);
    }

    public static void main(String[] args) throws Exception {
        System.out.println("Задание 7. Запись текста в файл и подсчёт
        СИМВОЛОВ");
        run();
    }
}

```

Задание 8 (класс `lr7.task8.Task8`):

```

package lr7.task8;

import lr7.FileDemoConfig;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serial;
import java.io.Serializable;
import java.nio.file.Files;

public class Task8 {

    public static class Student implements Serializable {

        @Serial
        private static final long serialVersionUID = 1L;

        private final String fullName;
        private final String group;
        private final double averageGrade;

        public Student(String fullName, String group, double averageGrade) {
            this.fullName = fullName;
            this.group = group;
            this.averageGrade = averageGrade;
        }

        public String getFullName() {
            return fullName;
        }
    }
}

```

```

        public String getGroup() {
            return group;
        }

        public double getAverageGrade() {
            return averageGrade;
        }

        @Override
        public String toString() {
            return "Student{fullName='" + fullName + "', group='" + group + "',
averageGrade=" + averageGrade + "}";
        }
    }

    public static void run() throws Exception {
        var file = FileDemoConfig.resolve("task8", "student.ser");
        Files.createDirectories(file.getParent());

        Student student = new Student("Владимиров Никита", "РИМ-150950", 4.75);
        System.out.println("Исходный объект: " + student);

        try (ObjectOutputStream output = new ObjectOutputStream(new
FileOutputStream(file.toFile())) {
            output.writeObject(student);
        }
        System.out.println("Объект записан в файл.");

        Student restored;
        try (ObjectInputStream input = new ObjectInputStream(new
FileInputStream(file.toFile())) {
            restored = (Student) input.readObject();
        }

        System.out.println("Восстановленный объект:");
        System.out.println("  ФИО: " + restored.getFullName());
        System.out.println("  Группа: " + restored.getGroup());
        System.out.println("  Средний балл: " + restored.getAverageGrade());

        Files.deleteIfExists(file);
    }

    public static void main(String[] args) throws Exception {
        System.out.println("Задание 8. Сериализация и десериализация объекта
Student");
        run();
    }
}

```

3.4. Класс FileDemoConfig

```

package lr7;

import java.nio.file.Path;
import java.nio.file.Paths;

public final class FileDemoConfig {

    private static final Path DATA_DIR =

```

```

Paths.get(System.getProperty("user.dir"), "data");

private FileDemoConfig() {
}

public static Path getDataDir() {
    return DATA_DIR;
}

public static Path resolve(String first, String... more) {
    return DATA_DIR.resolve(Paths.get(first, more));
}

public static boolean isDemoMode() {
    return Boolean.getBoolean("lab7.demo");
}
}

```

3.5. Класс Main

```

package lr7;

public class Main {

    public static void main(String[] args) throws Exception {
        System.out.println("=== Лабораторная работа №7: Ввод/вывод и работа с файлами ===\n");

        System.out.println("--- Примеры из раздела 1 ---\n");
        examples_lr7.Example1.main(args);
        System.out.println();

        examples_lr7.Example2.main(args);
        System.out.println();

        examples_lr7.Example3.main(args);
        System.out.println();

        examples_lr7.Example4.main(args);
        System.out.println();

        examples_lr7.Example5.main(args);
        System.out.println();

        examples_lr7.Example6.main(args);
        System.out.println();

        examples_lr7.Example7.main(args);
        System.out.println();

        examples_lr7.Example8.main(args);
        System.out.println();

        examples_lr7.Example9.main(args);
        System.out.println();

        System.out.println("--- Задания для самостоятельной работы ---\n");
        lr7.task3.Task3.main(args);
        System.out.println();
    }
}

```

```

        lr7.task4.Task4.main(args);
        System.out.println();

        lr7.task5.Task5.main(args);
        System.out.println();

        lr7.task6.Task6.main(args);
        System.out.println();

        lr7.task7.Task7.main(args);
        System.out.println();

        lr7.task8.Task8.main(args);
    }
}

```

3.6. Задачи Timus

Задача 1083: сумма целых чисел из входного потока.

```

package timus.task_1083;

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        long sum = 0;
        while (scanner.hasNextLong()) {
            sum += scanner.nextLong();
        }
        System.out.println(sum);
    }
}

```

Задача 1409 «Восьмой аргумент»: вычисление целочисленного частного степеней.

```

package timus.task_1409;

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        long a = scanner.nextLong();
        long b = scanner.nextLong();
        long c = scanner.nextLong();
        long d = scanner.nextLong();

        long numerator = pow(a, b);
        long denominator = pow(c, d);
    }
}

```

```

        System.out.println(numerator / denominator);
    }

    private static long pow(long base, long exponent) {
        long result = 1;
        for (long i = 0; i < exponent; i++) {
            result *= base;
        }
        return result;
    }
}

```

4. Вывод

В ходе выполнения лабораторной работы изучены механизмы ввода/вывода в Java: работа с классом File, байтовые и символьные потоки, буферизация, адаптеры потоков, PrintWriter и сериализация объектов. Реализованы все примеры и задания методички, включая операции с текстовыми файлами и сохранение объектов в бинарном виде. Программа успешно компилируется и выполняется. Дополнительно решены задачи 1083 и 1409 с сайта acm.timus.ru.

5. Результаты выполнения программы

Примечание: для заданий 2, 5, 6 и 7 использован демонстрационный режим (-Dlab7.demo=true), ввод с консоли подставляется автоматически.

=== Примеры из раздела 1 (классы Example1-Example9) ===

```

>>> java examples_lr7.Example1
Пример 1.1. Работа с классом File
Создание папки: .\data\example1\example_folder
Папка создана.
Создание файла: .\data\example1\example_folder\example_file.txt
Файл создан.
Имя файла: example_file.txt
Абсолютный путь: .\data\example1\example_folder\example_file.txt
Размер файла: 0 байт
Это файл: true
Это каталог: false
Удаление файла...
Файл удалён.
Удаление папки...
Папка удалена.

>>> java examples_lr7.Example2
Пример 1.2. Байтовый ввод и вывод (FileInputStream / FileOutputStream)
Запись данных в файл: "data"
Данные записаны.
Чтение данных из файла:
Прочитано: data
Удаление файла...

```

Файл удалён.

```
>>> java examples_lr7.Example3
Пример 1.3. Символьные потоки (FileReader / FileWriter)
Запись данных в файл: "data"
Данные записаны.
Чтение данных из файла:
Прочитано: data
Удаление файла...
Файл удалён.
```

```
>>> java examples_lr7.Example4
Пример 1.4. Буферизация (BufferedReader / BufferedWriter)
Запись данных в файл: "data"
Данные записаны.
Чтение данных из файла:
Прочитано: data
Удаление файла...
Файл удалён.
```

```
>>> java examples_lr7.Example5
Пример 1.5. Преобразование байтовых потоков в символьные
Исходный файл input.txt создан.
Чтение из input.txt и преобразование в верхний регистр...
Результат записан в output.txt:
ПЕРВАЯ СТРОКА
ВТОРАЯ СТРОКА
ТРЕТЬЯ СТРОКА
```

Временные файлы удалены.

```
>>> java examples_lr7.Example6
Пример 1.6. Класс PrintWriter
Запись в файл с помощью PrintWriter...
Содержимое файла:
Строка без перевода. Строка с переводом.
Число: 42, дробь: 3,14
```

```
Вывод на консоль через PrintWriter(System.out):
PŸPsPsP±C%PuPSPëPu C±PuCБPuP· PrintWriter
```

```
>>> java examples_lr7.Example7
Пример 1.7. Сериализация объекта Person
Исходный объект: Person{name='Иван', age=25}
Сериализация объекта в файл...
Десериализация объекта из файла...
Восстановленный объект: Person{name='Иван', age=25}
Файл сериализации удалён.
```

```
>>> java examples_lr7.Example8
Задача 3. Подсчёт строк в текстовом файле
Файл: .\data\example8\lines.txt
Количество строк: 3
```

```
>>> java examples_lr7.Example9
Задача 4. Копирование текстового файла
Исходный файл: .\data\example9\source.txt
Файл скопирован в: .\data\example9\target.txt
Содержимое копии:
```

Текст для копирования.
Вторая строка.

=== Задание 1. Воспроизведение примеров (lr7.task1.Task1) ===

```
>>> java lr7.task1.Task1
Задание 1. Воспроизведение всех примеров из раздела 1
Создание папки: .\data\example1\example_folder
Папка создана.
Создание файла: .\data\example1\example_folder\example_file.txt
Файл создан.
Имя файла: example_file.txt
Абсолютный путь: .\data\example1\example_folder\example_file.txt
Размер файла: 0 байт
Это файл: true
Это каталог: false
Удаление файла...
Файл удалён.
Удаление папки...
Папка удалена.
```

```
Запись данных в файл: "data"
Данные записаны.
Чтение данных из файла:
Прочитано: data
Удаление файла...
Файл удалён.
```

```
Запись данных в файл: "data"
Данные записаны.
Чтение данных из файла:
Прочитано: data
Удаление файла...
Файл удалён.
```

```
Запись данных в файл: "data"
Данные записаны.
Чтение данных из файла:
Прочитано: data
Удаление файла...
Файл удалён.
```

```
Исходный файл input.txt создан.
Чтение из input.txt и преобразование в верхний регистр...
Результат записан в output.txt:
ПЕРВАЯ СТРОКА
ВТОРАЯ СТРОКА
ТРЕТЬЯ СТРОКА
```

Временные файлы удалены.

```
Запись в файл с помощью PrintWriter...
Содержимое файла:
Строка без перевода. Строка с переводом.
Число: 42, дробь: 3,14
```

```
Вывод на консоль через PrintWriter(System.out):
PŲPsPsP±C%PµPSPĖPµ C†PµCЪPµP· PrintWriter
```

```
=== Задание 2. Ввод с консоли (lr7.task2.Task2) ===
Ввод: demo data from console (5 раз, для каждого типа потока)

>>> java -Dlab7.demo=true lr7.task2.Task2
Задание 2. Ввод данных с консоли для записи в файл
--- Байтовые потоки ---
Введите данные для записи в файл (демо): demo data from console
Прочитано из файла: demo data from console

--- Символьные потоки ---
Введите данные для записи в файл (демо): demo data from console
Прочитано из файла: demo data from console

--- Буферизованные потоки ---
Введите данные для записи в файл (демо): demo data from console
Прочитано из файла: demo data from console

--- Адаптеры потоков ---
Введите данные для записи в файл (демо): demo data from console
Преобразованный текст: DEMO DATA FROM CONSOLE

--- PrintWriter ---
Введите данные для записи в файл (демо): demo data from console
Записано в файл: demo data from console

=== Задания 3-8 ===

--- Задание 3 (lr7.task3.Task3) ---
>>> java lr7.task3.Task3
Задание 3. Подсчёт строк в текстовом файле
Файл: sample.txt
Количество строк: 3

--- Задание 4 (lr7.task4.Task4) ---
>>> java lr7.task4.Task4
Задание 4. Копирование текстового файла
Файл скопирован.
Содержимое исходного файла.
Ещё одна строка.

--- Задание 5 (lr7.task5.Task5) ---
Ввод: файл: task5/size_demo.txt
>>> java -Dlab7.demo=true lr7.task5.Task5
Задание 5. Размер файла в байтах
Введите имя файла (демо): task5/size_demo.txt
Размер файла "size_demo.txt": 69 байт

--- Задание 6 (lr7.task6.Task6) ---
Ввод: файл: task6/search.txt, слово: Java
>>> java -Dlab7.demo=true lr7.task6.Task6
Задание 6. Поиск строк, содержащих заданное слово
Введите имя файла (демо): task6/search.txt
Введите слово для поиска (демо): Java
Строки, содержащие "Java":
Java — язык программирования.
Работа с файлами в Java.

--- Задание 7 (lr7.task7.Task7) ---
```

Ввод: файл: task7/output.txt, текст: Текст для записи в файл.

```
>>> java -Dlab7.demo=true lr7.task7.Task7
```

Задание 7. Запись текста в файл и подсчёт символов

Введите имя файла (демо): task7/output.txt

Введите текст (демо): Текст для записи в файл.

Записано символов: 24

Содержимое файла: Текст для записи в файл.

--- Задание 8 (lr7.task8.Task8) ---

```
>>> java lr7.task8.Task8
```

Задание 8. Сериализация и десериализация объекта Student

Исходный объект: Student{fullName='Владимиров Никита', group='РИМ-150950', averageGrade=4.75}

Объект записан в файл.

Восстановленный объект:

 ФИО: Владимиров Никита

 Группа: РИМ-150950

 Средний балл: 4.75

=== Задачи Timus ===

Задача 1083. Ввод: 1 2 3

```
>>> java timus.task_1083.Main
```

6

Задача 1409. Ввод: 2 2 4 1

```
>>> java timus.task_1409.Main
```

1